DID 身份会话管理模块

微信小程序中的**DID 会话管理模块**。DID 是“去中心化标识符”的简称。这个模块主要做了两件事：

1. **检查用户的 DID 状态**：比如有没有创建过 DID、DID 是否在链上注册成功、链下文档是否完整、本机是否保存了私钥等。
2. **跳转与清理**：根据检查结果，自动跳转到登录页或提示用户进行数据恢复；也提供了清除本地身份信息的方法。

代码里大量使用了微信小程序的 API，比如 `wx.getStorageSync` 读写本地缓存，`wx.request` 发网络请求，`wx.reLaunch` 关闭所有页面打开新页面，以及 `getCurrentPages` 获取页面栈。

```
const { baseURL } = require('./config');

const LOGIN_URL = '/pages/login/login';

// 防止重复跳转登录页的标志
let isRedirectingToLogin = false;

/**
 * 获取当前页面的路由路径。
 * 利用微信的页面栈，返回最顶层页面的带斜杠路由，比如 "/pages/index/index"。
 * @returns {string} 当前页面路由，如果获取不到则返回空字符串。
 */
function getCurrentRoute() {
  const pages = getCurrentPages();
  const currentPage = pages[pages.length - 1];

  if (!currentPage || !currentPage.route) {
    return '';
  }

  return '/' + currentPage.route;
}

/**
 * 跳转到登录页面。
 * 如果当前已在登录页，或正在跳转中，则不再重复跳转。
 * 可以显示一条短暂的 Toast 提示，之后会关闭所有页面并打开登录页。
 * @param {string} [message] - 要显示的提示文字，没有则不显示。
 */
function redirectToLogin(message) {
  const currentRoute = getCurrentRoute();

  // 已在登录页，无需跳转
  if (currentRoute === LOGIN_URL) {
    return;
  }

  // 正在跳转中，避免重复触发
  if (isRedirectingToLogin) {
    return;
```

```
  }

  isRedirectingToLogin = true;

  if (message) {
    wx.showToast({
      title: message,
      icon: 'none',
      duration: 1000
    });
  }

  // 设置一个延迟，确保 Toast 能完整显示再跳转
  setTimeout(() => {
    wx.reLaunch({
      url: LOGIN_URL,
      complete: () => {
        // 跳转完成后重置标志，允许下次再跳转
        isRedirectingToLogin = false;
      }
    });
  }, message ? 700 : 0);
}

/**
 * 生成存储 DID 私钥的本地缓存 key。
 * 格式：didPrivateKey:<did>
 * @param {string} did - DID 字符串。
 * @returns {string} 存储用的 key。
 */
function getPrivateKeyStorageKey(did) {
  return `didPrivateKey:${did}`;
}

/**
 * 判断本地是否保存了指定 DID 的私钥。
 * @param {string} did - DID 字符串。
 * @returns {boolean} 如果有私钥则返回 true，否则 false。
 */
function hasDidPrivateKeyLocal(did) {
  if (!did) return false;
  return !!wx.getStorageSync(getPrivateKeyStorageKey(did));
}

/**
 * 清除当前 DID 的会话信息（不包括私钥）。
 * 会删除本地存储的 currentDid 和 currentDidInfo。
 * 通常用于“退出当前绑定但保留私钥”的场景。
 */
function clearDidSession() {
  wx.removeStorageSync('currentDid');
  wx.removeStorageSync('currentDidInfo');
}

/**
 * 清除当前 DID 的会话信息，同时删除本机保存的该 DID 私钥。
```

```
 * 适用于用户主动选择“删除身份和私钥”的彻底清理场景。
 */
function clearDidSessionWithPrivateKey() {
  const currentDid = wx.getStorageSync('currentDid') || '';

  clearDidSession();

  if (currentDid) {
    wx.removeStorageSync(getPrivateKeyStorageKey(currentDid));
  }
}

/**
 * 本地 DID 状态检查（不联网）。
 * 只检查：
 * 1. 本地是否有 currentDid；
 * 2. 本地是否有对应的私钥。
 * 适用于“我的”页面展示本地状态、判断是否拥有控制权等场景。
 *
 * @param {Object} [options] - 配置项。
 * @param {boolean} [options.redirect=true] - 没有 did 时是否自动跳转登录页。
 * @param {function} [options.success] - 检查通过后的回调，参数 { currentDid,
hasPrivateKey }。
 * @param {function} [options.fail] - 检查失败（无 did）的回调，参数 { reason:
'NO_CURRENT_DID' }。
 */
function checkDidLocal(options = {}) {
  const {
    redirect = true,
    success,
    fail
  } = options;

  const currentDid = wx.getStorageSync('currentDid') || '';

  if (!currentDid) {
    if (redirect) {
      redirectToLogin('请先创建或导入 DID');
    }

    if (typeof fail === 'function') {
      fail({
        reason: 'NO_CURRENT_DID'
      });
    }

    return;
  }

  const hasPrivateKey = hasDidPrivateKeyLocal(currentDid);

  if (typeof success === 'function') {
    success({
      currentDid,
      hasPrivateKey
    });
```

```
  }
}

/**
 * 检查 DID 是否在链上可达（联网）。
 * 会向后端发送请求，检查 DID 是否已上链。即使链下文档缺失，也不会清除本地 DID，
 * 因为文档缺失是一种需要被恢复的异常状态。
 * 适用于首页、操作页、状态查询、数据恢复等场景。
 *
 * @param {Object} [options] - 配置项。
 * @param {string} [options.did] - 要检查的 DID；不传则使用本地存储的 currentDid。
 * @param {boolean} [options.redirect=true] - 出现无 DID 或链上不存在时是否跳转登录。
 * @param {function} [options.success] - 检查通过的回调，参数包含：
 *   currentDid, hasPrivateKey, didInfo, chainState, currentDocument,
documentMissing。
 * @param {function} [options.fail] - 检查失败的回调，参数包含 reason 和可能的
response/error。
 */
function checkDidReachable(options = {}) {
  const {
    did,
    redirect = true,
    success,
    fail
  } = options;

  const targetDid = did || wx.getStorageSync('currentDid') || '';

  if (!targetDid) {
    if (redirect) {
      redirectToLogin('请先创建或导入 DID');
    }

    if (typeof fail === 'function') {
      fail({
        reason: 'NO_CURRENT_DID'
      });
    }

    return;
  }

  wx.request({
    url: baseURL + '/api/did/latest',
    method: 'GET',
    data: {
      did: targetDid
    },
    header: {
      'content-type': 'application/json'
    },
    success: (res) => {
      const payload = res.data || {};

      // 网络请求失败或后端返回 success: false
```

```
      if (res.statusCode < 200 || res.statusCode >= 300 || payload.success ===
false) {
        if (redirect) {
          redirectToLogin(payload.message || 'DID 状态异常');
        }

        if (typeof fail === 'function') {
          fail({
            reason: 'REQUEST_FAILED',
            response: payload
          });
        }

        return;
      }

      const data = payload.data || payload;
      const chainState = data.chain_state || data.chainState || {};
      const currentDocument = data.current_document || data.currentDocument ||
null;

      // 如果后端明确返回 exists: false，则链上不存在；否则认为存在
      const chainExists = chainState.exists !== false;

      if (!chainExists) {
        if (redirect) {
          redirectToLogin('DID 未在链上注册');
        }

        if (typeof fail === 'function') {
          fail({
            reason: 'DID_NOT_ON_CHAIN',
            response: data
          });
        }

        return;
      }

      const hasPrivateKey = hasDidPrivateKeyLocal(targetDid);
      const documentMissing = !currentDocument;

      // 更新本地存储的 DID 信息
      wx.setStorageSync('currentDid', targetDid);
      wx.setStorageSync('currentDidInfo', data);

      if (typeof success === 'function') {
        success({
          currentDid: targetDid,
          hasPrivateKey,
          didInfo: data,
          chainState,
          currentDocument,
          documentMissing
        });
      }
```

```
    },
    fail: (err) => {
      if (redirect) {
        wx.showToast({
          title: '网络请求失败',
          icon: 'none'
        });
      }

      if (typeof fail === 'function') {
        fail({
          reason: 'NETWORK_ERROR',
          error: err
        });
      }
    }
  });
}

/**
 * 严格锁定 DID 可用状态。
 * 要求同时满足：本地有 DID、链上注册成功、链下文档存在。
 * 如果链下文档缺失，会弹窗提示用户前往数据恢复页面，而不是直接跳转登录。
 * 适用于 DID 更新、身份认证、属性证明等需要完整 DID 文档的操作。
 *
 * @param {Object} [options] - 配置项。
 * @param {boolean} [options.redirect=true] - 出现前置失败时是否跳转或弹窗。
 * @param {function} [options.success] - 检查完全通过的回调，参数与 checkDidReachable
类似，但不含 documentMissing。
 * @param {function} [options.fail] - 任何失败的回调，参数包含 reason 及可能的
currentDid 等信息。
 */
function checkDidReady(options = {}) {
  const {
    redirect = true,
    success,
    fail
  } = options;

  checkDidReachable({
    redirect,
    success: ({ currentDid, hasPrivateKey, didInfo, chainState, currentDocument,
documentMissing }) => {
      if (documentMissing) {
        if (redirect) {
          wx.showModal({
            title: '链下文档缺失',
            content: '当前 DID 已在链上注册，但链下 DID 文档缺失。请先进行数据恢复。',
            confirmText: '去恢复',
            cancelText: '取消',
            success: (modalRes) => {
              if (modalRes.confirm) {
                wx.navigateTo({
                  url: `/pages/didRecover/didRecover?
did=${encodeURIComponent(currentDid)}`
                });
```

```
            }
          }
        });
      }

      if (typeof fail === 'function') {
        fail({
          reason: 'CURRENT_DOCUMENT_MISSING',
          currentDid,
          hasPrivateKey,
          didInfo,
          chainState
        });
      }

      return;
    }

    if (typeof success === 'function') {
      success({
        currentDid,
        hasPrivateKey,
        didInfo,
        chainState,
        currentDocument
      });
    }
  },
  fail
});
}

/**
 * 检查本机是否拥有指定 DID 的控制私钥。
 * 这是纯本地检查，真正的控制权认证需要在后续通过”挑战-响应“流程完成。
 * 适用于点击”更新 DID“、”数据恢复“等关键操作前的权限校验。
 *
 * @param {Object} [options] - 配置项。
 * @param {string} [options.did] - 要检查的 DID，不传则使用本地 currentDid。
 * @param {boolean} [options.showToast=true] - 当没有私钥时是否显示 Toast 提示。
 * @param {function} [options.success] - 检查通过的回调，参数 { currentDid }。
 * @param {function} [options.fail] - 检查失败的回调，参数 { reason, currentDid? }。
 */
function checkDidController(options = {}) {
  const {
    did,
    showToast = true,
    success,
    fail
  } = options;

  const targetDid = did || wx.getStorageSync('currentDid') || '';

  if (!targetDid) {
    if (showToast) {
      wx.showToast({
```

```
        title: '请先创建或导入 DID',
        icon: 'none'
      });
    }

    if (typeof fail === 'function') {
      fail({
        reason: 'NO_CURRENT_DID'
      });
    }

    return;
  }

  if (!hasDidPrivateKeyLocal(targetDid)) {
    if (showToast) {
      wx.showToast({
        title: '当前设备没有该 DID 私钥',
        icon: 'none'
      });
    }

    if (typeof fail === 'function') {
      fail({
        reason: 'NO_DID_PRIVATE_KEY',
        currentDid: targetDid
      });
    }

    return;
  }

  if (typeof success === 'function') {
    success({
      currentDid: targetDid
    });
  }
}

/**
 * 兼容旧版 checkDidSession 调用。
 * 旧页面中如果调用了 checkDidSession，会直接执行 checkDidReady（严格检查）。
 * 建议未来逐步将调用替换为更明确的 checkDidReachable、checkDidReady 或
checkDidController。
 *
 * @param {Object} [options] - 配置项，直接传递给 checkDidReady。
 */
function checkDidSession(options = {}) {
  checkDidReady(options);
}

/**
 * 获取当前存储的 DID 字符串。
 * @returns {string} 当前 DID，如果没有则返回空字符串。
 */
function getCurrentDid() {
```

```
  return wx.getStorageSync('currentDid') || '';
}

/**
 * 设置当前 DID 并可选地保存 DID 详细信息。
 * @param {string} did - DID 字符串。
 * @param {Object} [didInfo] - DID 相关的详情数据（如链状态、文档等）。
 */
function setCurrentDid(did, didInfo) {
  wx.setStorageSync('currentDid', did);

  if (didInfo) {
    wx.setStorageSync('currentDidInfo', didInfo);
  }
}

/**
 * 获取当前 DID 的详细信息。
 * @returns {Object|null} 存储的 DID 详情对象，如果没有则返回 null。
 */
function getCurrentDidInfo() {
  return wx.getStorageSync('currentDidInfo') || null;
}

module.exports = {
  checkDidLocal,
  checkDidReachable,
  checkDidReady,
  checkDidController,
  checkDidSession,

  clearDidSession,
  clearDidSessionWithPrivateKey,

  getCurrentDid,
  getCurrentDidInfo,
  setCurrentDid
};
```

第一性原理：证明“我拥有私钥”而不泄露私钥

你不能直接把私钥发给服务器（那就泄露了），而是要用一个经典的密码学流程：

挑战-响应（Challenge-Response）

1. **服务器向你发出一个“挑战”**：一个随机数（Nonce），比如“982156”。
2. **你在本地用私钥对这个随机数签名**，生成一段签名数据。
3. **你把签名结果发回给服务器。**
4. **服务器用你事先公布的公钥（链上公开的）验证签名。**如果验证通过，就证明你确实持有对应私钥，而没有泄露私钥本身。

这一步在代码中虽然没完整实现，但在 `checkDidController` 的注释中明确提到了：

> 真正的控制权认证仍然要通过： challenge -> 本地签名 -> 后端验签。

所以整个系统最底层的信任根基，就是这个**私钥签名验证。**

从“打开小程序”到“完成身份认证”的全流程拆解

第一阶段：检查本地有没有 DID 和私钥（离线）

执行函数	`checkDidLocal`
做了什么	不看网络，只翻本地存储（手机缓存）。
检查项	1. 有没有 `currentDid`（当前使用的 DID 字符串）？ 2. 有没有 `didPrivateKey:<did>`（对应的私钥）？
结果	✅ **都有**：进入下一步。 ❌ **没有** `currentDid`：直接跳转登录页，让用户创建或导入 DID。 ❌ **有 DID 但无私钥**：说明这个 DID 不是本机创建的，用户无法控制它。

第二阶段：检查 DID 是否真实存在于区块链上（联网）

执行函数	`checkDidReachable`
做了什么	拿着 `currentDid` 去调后端接口 `/api/did/latest`，询问这个 DID 是否已经上链。
检查项	1. 链上状态 `chainState.exists` 是否为 `true`（不是 `false`）。
结果	✅ **链上存在**：进入下一步，并更新本地存储的最新 DID 信息。 ❌ **链上不存在**：跳转登录页，提示“DID 未在链上注册”。

> **特别注意**：这一步如果后端返回“链下文档缺失”（`documentMissing=true`），它 **不会** 判定失败，而是继续往下走。因为“链上存在但链下文档丢了”是一个需要用户去修复的中间状态，不能直接踢回登录页。

第三阶段：检查 DID 是否完全就绪（链上存在 + 链下文档完整）

执行函数	`checkDidReady`
做了什么	在 `checkDidReachable` 成功的基础上，进一步要求“链下文档”（current_document）必须存在。
检查项	`documentMissing` 是否为 `false`。
结果	✅ **文档完整**：进入最终认证环节。 ❌ **文档缺失**：弹出模态框，告知用户“链下 DID 文档缺失，请先进行数据恢复”，并提供“去恢复”按钮。

第四阶段：检查本机是否持有控制私钥（操作前的最后守卫）

执行函数	`checkDidController`
做了什么	纯本地检查，再次确认 `didPrivateKey:<did>` 确实存在。

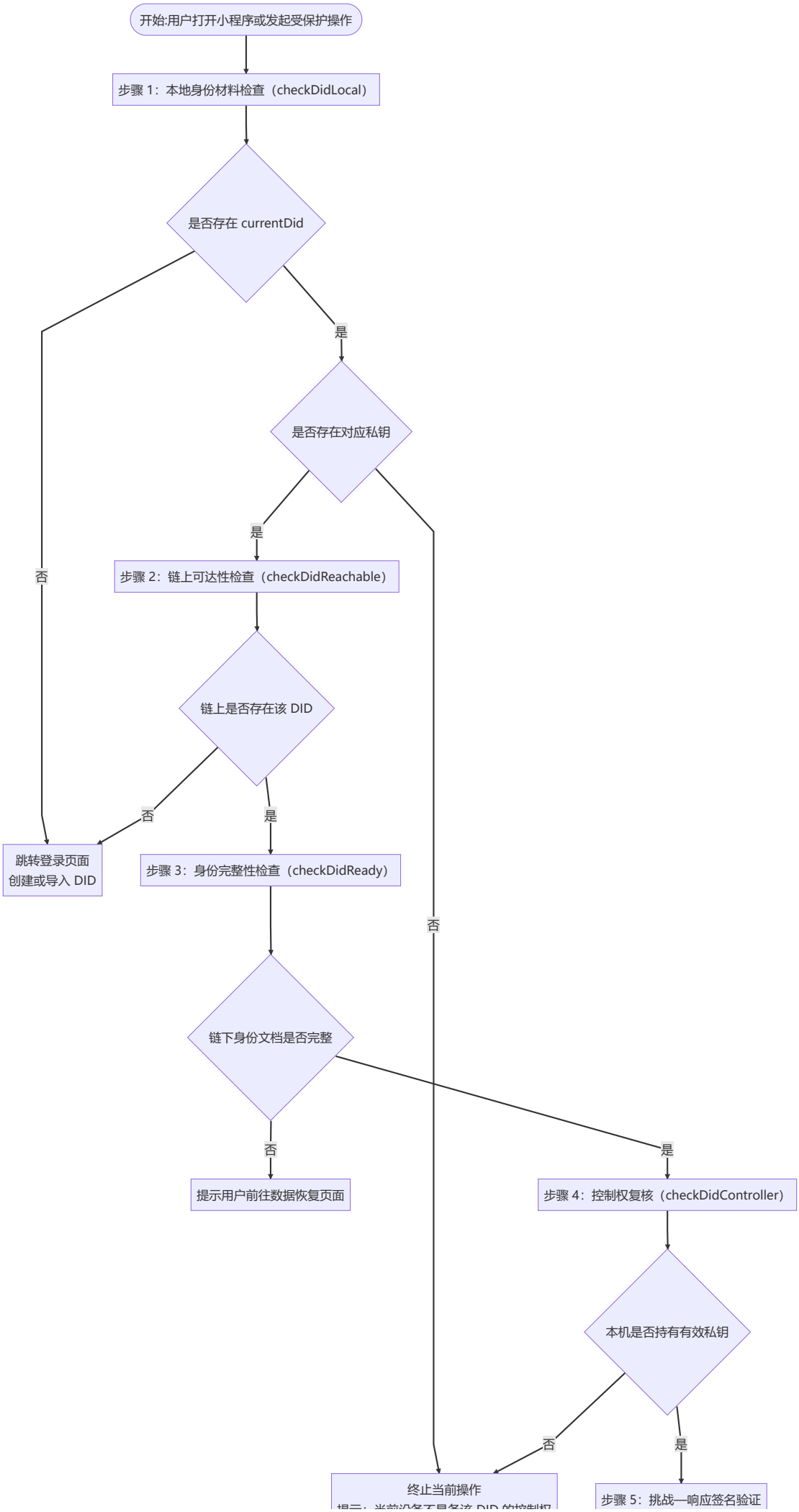
执行函数	`checkDidController`
检查项	`hasDidPrivateKeyLocal(targetDid)` 是否为 `true`。
结果	✅ **有私钥**：允许进入后续的关键操作（如更新 DID、签名认证）。 ❌ **无私钥**：toast 提示“当前设备没有该 DID 私钥”，操作被拦截。

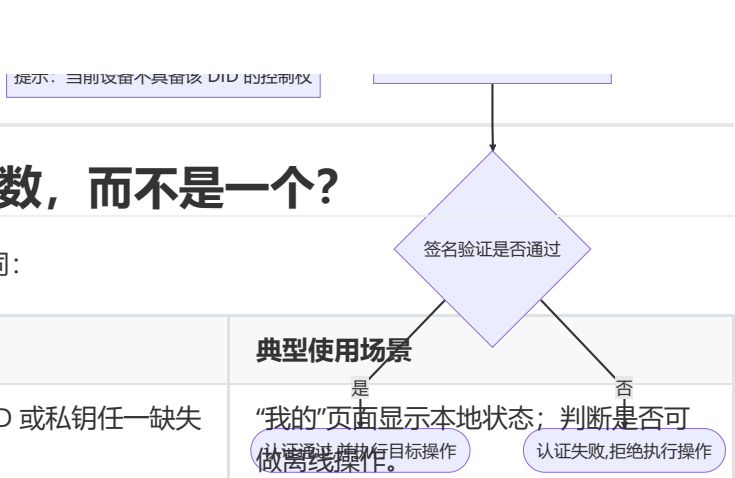
第五阶段：真正的挑战-响应认证（代码注释暗示的下一步）

前面四步都是“资格检查”，最终的安全验证是在第五步完成的，一般流程如下：

1. **用户点击“认证”或“签名操作”。**
2. **前端调用后端接口获取一个随机挑战字符串（Nonce）。**
3. **前端用本地私钥对 Nonce 签名**（这部分代码可能在小程序的其他工具函数里）。
4. **把签名结果和 DID 一起发给后端。**
5. **后端从链上获取该 DID 的公钥，验证签名。**验证通过，则返回认证成功凭证（如 JWT Token）。

整个认证流程可以用下图清晰表示：


开始:用户打开小程序或发起受保护操作
步骤 1：本地身份材料检查（checkDidLocal）
是否存在 currentDid
否
是
是否存在对应私钥
是
否
步骤 2：链上可达性检查（checkDidReachable）
链上是否存在该 DID
否
是
跳转登录页面
创建或导入 DID
步骤 3：身份完整性检查（checkDidReady）
否
链下身份文档是否完整
否
是
提示用户前往数据恢复页面
步骤 4：控制权复核（checkDidController）
本机是否持有有效私钥
否
是
终止当前操作
步骤 5：挑战—响应签名验证



为什么需要四个检查函数，而不是一个？

因为它们各自的**容错级别**和**使用场景**不同：

函数	容错级别	典型使用场景
`checkDidLocal`	最低。DID 或私钥任一缺失即失败。	“我的”页面显示本地状态；判断是否可做离线操作。
`checkDidReachable`	中等。允许链下文档暂时缺失。	首页、数据恢复页（文档缺失正是要修复的异常）。
`checkDidReady`	最高。要求文档完整。	DID 更新、身份认证等需要完整 DID 信息的操作。
`checkDidController`	单点检查。只关注私钥是否存在。	任何写入链上数据的操作前，确保本机能签名。

这种分层设计既保证了安全，又不会因为部分链下数据临时缺失而把用户完全拒之门外，给了修复的机会。

补充：登录与 DID 创建/导入

代码中最开始的 `redirectToLogin` 是所有检查失败的统一归宿。这个登录页通常不是用户名密码登录，而是引导用户：

- **创建新 DID**：生成一对公私钥，把公钥和 DID 文档注册到链上，私钥加密存储到本地。
- **导入已有 DID**：用户提供私钥或助记词，恢复出私钥并存储到本地。

所以整个流程的闭环是：**没有 DID → 登录创建/导入 → 本地拥有私钥 → 通过层层检查 → 用私钥签名证明身份。**

总结

这套 DID 身份认证流程从第一性原理出发，核心就是 **“私钥签名验证”** 。在它外面，包裹了多层本地与链上的状态检查，确保每次操作都是在 **“身份真实、文档完整、本机可控”** 的前提下进行的。这样既保证了安全性，又对数据恢复等边界情况做了友好的用户体验设计。

私钥生成模块

这是一个**微信小程序里的 DID 密钥管理工具**，相当于一个“钥匙串”。DID 是去中心化身份标识符，这里用 `did:ethr:<以太坊地址>` 格式。模块核心功能包括：

1. **生成密钥对**：在本地创建 secp256k1 椭圆曲线上的私钥，并推导出以太坊地址和 DID。
2. **签名认证**：用私钥对消息进行符合以太坊 `personal_sign` 规范的签名，用于“挑战-响应”身份验证。
3. **存储与管理**：保存、读取、删除指定 DID 的私钥到微信本地缓存，并检查私钥是否属于某个 DID。

代码避免使用在小程序中容易出问题的 ethers.js 或 @noble/secp256k1，而是采用 `elliptic` 做椭圆曲线运算，`js-sha3` 做哈希，轻量可靠。

```
/**
 * DID 密钥管理工具模块（微信小程序端）
 *
 * 当前实现不使用 ethers / @noble/secp256k1。
 * 原因：
 * - ethers 在微信小程序环境中会触发 Buffer/process/elliptic 兼容问题。
 * - @noble/secp256k1 在小程序构建后会尝试 require('crypto')，导致黑屏。
 *
 * 当前实现使用：
 * - elliptic：secp256k1 椭圆曲线签名
 * - js-sha3：keccak256 哈希
 *
 * 签名格式兼容以太坊 personal_sign：
 * hash = keccak256("\x19Ethereum Signed Message:\n" + len(message) + message)
 * signature = r + s + v
 */

const elliptic = require('elliptic');
const { keccak_256 } = require('js-sha3');

const EC = elliptic.ec;
const ec = new EC('secp256k1');

/**
 * 把字节数组转换为十六进制字符串，不带 `0x` 前缀。
 * 比如 [0xab, 0x0c] → "ab0c"。
 * @param {Uint8Array} bytes - 字节数组。
 * @returns {string} 十六进制字符串。
 */
function bytesToHex(bytes) {
  return Array.prototype.map.call(bytes, (byte) => {
    return byte.toString(16).padStart(2, '0');
  }).join('');
}

/**
 * 把十六进制字符串转为 Uint8Array。
 * 会自动去掉可能存在的 `0x` 前缀，并处理奇数长度补零。
 * @param {string} hex - 十六进制字符串（可选带 0x）。
 * @returns {Uint8Array}
 */
function hexToBytes(hex) {
  let value = String(hex || '').trim();

  if (value.startsWith('0x')) {
    value = value.slice(2);
  }

  if (value.length % 2 !== 0) {
    value = '0' + value;
  }
```

```
  const bytes = new Uint8Array(value.length / 2);

  for (let i = 0; i < bytes.length; i++) {
    bytes[i] = parseInt(value.slice(i * 2, i * 2 + 2), 16);
  }

  return bytes;
}

/**
 * 把普通字符串按照 UTF-8 编码转为字节数组。
 * 优先使用 TextEncoder，如果环境不支持则用 encodeURIComponent 转义方式。
 * @param {string} str - 待转换的字符串。
 * @returns {Uint8Array} UTF-8 编码的字节数组。
 */
function utf8ToBytes(str) {
  const text = String(str || '');

  if (typeof TextEncoder !== 'undefined') {
    return new TextEncoder().encode(text);
  }

  // 小程序可能没有 TextEncoder，用传统方式
  const encoded = unescape(encodeURIComponent(text));
  const bytes = new Uint8Array(encoded.length);

  for (let i = 0; i < encoded.length; i++) {
    bytes[i] = encoded.charCodeAt(i);
  }

  return bytes;
}

/**
 * 拼接多个 Uint8Array，返回一个新的连续数组。
 * 用于将前缀消息和正文合并。
 * @param {...Uint8Array} arrays - 任意数量的字节数组。
 * @returns {Uint8Array}
 */
function concatBytes() {
  const arrays = Array.prototype.slice.call(arguments);
  const totalLength = arrays.reduce((sum, arr) => sum + arr.length, 0);
  const result = new Uint8Array(totalLength);

  let offset = 0;

  arrays.forEach((arr) => {
    result.set(arr, offset);
    offset += arr.length;
  });

  return result;
}

/**
 * 计算 keccak256 哈希，返回 ArrayBuffer。
```

```
 * 供 keccak256Bytes 内部使用，获取原始字节。
 * @param {Uint8Array} bytes - 要哈希的字节数组。
 * @returns {ArrayBuffer}
 */
function keccak256Hex(bytes) {
  return keccak_256.arrayBuffer(bytes);
}

/**
 * 计算 keccak256 哈希，返回 Uint8Array。
 * @param {Uint8Array} bytes - 要哈希的字节数组。
 * @returns {Uint8Array}
 */
function keccak256Bytes(bytes) {
  return new Uint8Array(keccak256Hex(bytes));
}

/**
 * 检查一个十六进制私钥是否在 secp256k1 曲线的有效范围内。
 * 必须满足 1 <= 私钥 < 曲线阶。
 * @param {string} hex - 十六进制私钥字符串（可带 0x）。
 * @returns {boolean}
 */
function isValidSecp256k1PrivateKey(hex) {
  try {
    const cleanHex = String(hex || '').replace(/^0x/, '');

    if (!/^[0-9a-fA-F]{64}$/.test(cleanHex)) {
      return false;
    }

    const bn = ec.keyFromPrivate(cleanHex, 'hex').getPrivate();

    return bn.cmpn(0) > 0 && bn.cmp(ec.curve.n) < 0;
  } catch (e) {
    return false;
  }
}

/**
 * 生成指定长度的随机字节。
 *
 * 注意：
 * 微信小程序的 `wx.getRandomValues` 不是标准浏览器 API，直接调用容易在模拟器里异常暂停。
 * 这里按优先级尝试：
 * 1. 标准 crypto.getRandomValues
 * 2. 微信 wx.getRandomValues 的对象式调用
 * 3. Math.random 兜底
 *
 * 毕设原型可以先用兜底方案跑通；
 * 正式生产环境应接入系统安全随机源或钱包。
 * @param {number} length - 字节长度。
 * @returns {Uint8Array}
 */
function randomBytes(length) {
  const bytes = new Uint8Array(length);
```

```
  // 1. 浏览器标准 crypto.getRandomValues
  try {
    const cryptoObj =
      typeof globalThis !== 'undefined'
        ? (globalThis.crypto || globalThis.msCrypto)
        : null;

    if (cryptoObj && typeof cryptoObj.getRandomValues === 'function') {
      cryptoObj.getRandomValues(bytes);
      return bytes;
    }
  } catch (e) {
    console.warn('[crypto] global crypto.getRandomValues unavailable:', e);
  }

  // 2. 微信小程序 wx.getRandomValues 兼容尝试
  try {
    if (typeof wx !== 'undefined' && typeof wx.getRandomValues === 'function') {
      const result = wx.getRandomValues({
        length
      });

      const buffer =
        result &&
        (result.randomValues || result.arrayBuffer || result.buffer || result);

      if (buffer instanceof ArrayBuffer) {
        const view = new Uint8Array(buffer);
        bytes.set(view.slice(0, length));
        return bytes;
      }

      if (buffer && buffer.byteLength) {
        const view = new Uint8Array(buffer);
        bytes.set(view.slice(0, length));
        return bytes;
      }
    }
  } catch (e) {
    console.warn('[crypto] wx.getRandomValues unavailable:', e);
  }

  // 3. 原型兜底：Math.random
  for (let i = 0; i < length; i++) {
    bytes[i] = Math.floor(Math.random() * 256);
  }

  return bytes;
}

/**
 * 生成一个 32 字节的 secp256k1 私钥（十六进制，带 0x 前缀）。
 * 会反复尝试直到生成在有效范围内的私钥，最多尝试 32 次防止死循环。
 * @returns {string} 私钥十六进制字符串，例如
"0x4c0883a69102937d6231471b5dbb6204fe5129617082792ae468d01a3f362318"
```

```
 */
function createPrivateKey() {
  const maxAttempts = 32;

  for (let i = 0; i < maxAttempts; i++) {
    const bytes = randomBytes(32);
    const hex = bytesToHex(bytes);
    const privateKey = '0x' + hex;

    if (isValidSecp256k1PrivateKey(privateKey)) {
      return privateKey;
    }
  }

  throw new Error('生成有效 secp256k1 私钥失败');
}

/**
 * 将私钥标准化为带 0x 前缀的 64 位小写十六进制。
 * 如果传入无效格式会抛出错误。
 * @param {string} privateKey - 私钥字符串。
 * @returns {string} 标准化后的私钥，如 "0x4c08...3218"。
 */
function normalizePrivateKey(privateKey) {
  const value = String(privateKey || '').trim();

  if (!value) {
    throw new Error('缺少私钥');
  }

  const hex = value.startsWith('0x') ? value.slice(2) : value;

  if (!/^[0-9a-fA-F]{64}$/.test(hex)) {
    throw new Error('私钥格式错误，应为 32 字节十六进制字符串');
  }

  return '0x' + hex.toLowerCase();
}

/**
 * 根据私钥生成 elliptic 库的 keyPair 对象，用于后续签名。
 * @param {string} privateKey - 标准化后的私钥。
 * @returns {elliptic.KeyPair}
 */
function getKeyPairFromPrivateKey(privateKey) {
  const normalizedPrivateKey = normalizePrivateKey(privateKey);
  return ec.keyFromPrivate(normalizedPrivateKey.slice(2), 'hex');
}

/**
 * 将以太坊地址转换为 EIP-55 校验和格式（混合大小写）。
 * 原理：取地址小写形式的 keccak256 哈希，若某位哈希值 >= 8，则对应地址字符大写。
 * @param {string} address - 20 字节地址（可带 0x）。
 * @returns {string} 带校验和的地址，如 "0xAb5801a7D398351b8bE11C439e05C5B3259aeC9B"
 */
function toChecksumAddress(address) {
```

```
  const raw = String(address || '').toLowerCase().replace(/^0x/, '');

  if (!/^[0-9a-f]{40}$/.test(raw)) {
    throw new Error('以太坊地址格式错误');
  }

  const hash = keccak_256(raw);
  let result = '0x';

  for (let i = 0; i < raw.length; i++) {
    result += parseInt(hash[i], 16) >= 8 ? raw[i].toUpperCase() : raw[i];
  }

  return result;
}

/**
 * 根据私钥推导以太坊地址（EIP-55 格式）。
 * 计算流程：
 * 1. 获取未压缩公钥（65 字节，04 + x + y）
 * 2. 去掉开头的 04 前缀，剩下 64 字节
 * 3. 对公钥做 keccak256 哈希
 * 4. 取最后 20 字节作为地址
 * 5. 转为校验和格式
 * @param {string} privateKey - 私钥十六进制字符串。
 * @returns {string} 以太坊校验和地址。
 */
function getAddressFromPrivateKey(privateKey) {
  const keyPair = getKeyPairFromPrivateKey(privateKey);
  const publicKeyArray = keyPair.getPublic(false, 'array'); // 65 bytes, 04 + x +
y
  const publicKeyWithoutPrefix = new Uint8Array(publicKeyArray.slice(1));
  const hash = keccak256Bytes(publicKeyWithoutPrefix);
  const addressBytes = hash.slice(hash.length - 20);

  return toChecksumAddress('0x' + bytesToHex(addressBytes));
}

/**
 * 生成一套全新的 DID 控制密钥对。
 * @returns {{ privateKey: string, address: string, did: string }}
 *          包含私钥、以太坊地址和 did:ethr 格式的 DID。
 */
function createDidKeyPair() {
  const privateKey = createPrivateKey();
  const address = getAddressFromPrivateKey(privateKey);

  return {
    privateKey,
    address,
    did: `did:ethr:${address}`
  };
}

/**
 * 从 did:ethr:<address> 格式的 DID 中提取出以太坊地址。
```

```
 * @param {string} did - 标准 DID 字符串。
 * @returns {string} 提取的以太坊地址（带 0x）。
 */
function extractAddressFromDid(did) {
  const prefix = 'did:ethr:';

  if (!did || !did.startsWith(prefix)) {
    throw new Error('DID格式必须为 did:ethr:<address>');
  }

  const address = did.slice(prefix.length);

  if (!address || !address.startsWith('0x')) {
    throw new Error('DID中缺少有效的以太坊地址');
  }

  return address;
}

/**
 * 将地址转为小写，用于忽略大小写的比较。
 * @param {string} address
 * @returns {string}
 */
function normalizeAddress(address) {
  return String(address || '').toLowerCase();
}

/**
 * 验证给定私钥是否属于指定的 DID。
 * 通过从私钥推导出地址，与从 DID 中提取的地址比对（忽略大小写）。
 * @param {string} did - did:ethr 字符串。
 * @param {string} privateKey - 私钥。
 * @returns {boolean} 是否匹配。
 */
function validatePrivateKeyForDid(did, privateKey) {
  const didAddress = extractAddressFromDid(did);
  const privateKeyAddress = getAddressFromPrivateKey(privateKey);

  return normalizeAddress(didAddress) === normalizeAddress(privateKeyAddress);
}

/**
 * 计算以太坊 personal_sign 规范的消息哈希。
 * 格式：keccak256("\x19Ethereum Signed Message:\n" + 消息字节长度 + 消息)
 * @param {string} message - 原始消息字符串（会被 UTF-8 编码）。
 * @returns {Uint8Array} 32 字节的消息哈希。
 */
function getEthereumMessageHash(message) {
  const messageBytes = utf8ToBytes(message);
  const prefixBytes = utf8ToBytes(`\x19Ethereum Signed
Message:\n${messageBytes.length}`);

  return keccak256Bytes(concatBytes(prefixBytes, messageBytes));
}
```

```
/**
 * 将十六进制字符串左侧补零到指定长度（用于签名 r、s 值标准化）。
 * @param {string} hex - 原始十六进制字符串（不带 0x）。
 * @param {number} length - 目标长度。
 * @returns {string}
 */
function padHex(hex, length) {
  return String(hex || '').padStart(length, '0');
}

/**
 * 使用私钥对 challenge 消息进行签名，返回以太坊格式的签名。
 *
 * 签名流程：
 * 1. 计算 personal_sign 消息哈希
 * 2. 使用椭圆曲线签名，得到 r, s, recoveryParam
 * 3. 将 r, s 填充到 64 位十六进制
 * 4. 计算 v = recoveryParam + 27
 * 5. 拼接为 0x + r + s + v 的 130 字符十六进制字符串
 *
 * 这个签名格式兼容以太坊的 eth_account.recover_message，后端可以用 v, r, s 恢复签名者地址。
 *
 * @param {string} privateKey - 私钥。
 * @param {string} challenge - 要签名的挑战消息（通常是随机字符串）。
 * @returns {Promise<string>} 以太坊格式的签名（0x + 130 个十六进制字符）。
 */
async function signChallenge(privateKey, challenge) {
  if (!challenge) {
    throw new Error('缺少挑战消息，无法签名');
  }

  const keyPair = getKeyPairFromPrivateKey(privateKey);
  const messageHash = getEthereumMessageHash(challenge);
  const messageHashHex = bytesToHex(messageHash);

  const signature = keyPair.sign(messageHashHex, {
    canonical: true
  });

  const r = padHex(signature.r.toString('hex'), 64);
  const s = padHex(signature.s.toString('hex'), 64);
  const recoveryParam = signature.recoveryParam;

  if (recoveryParam === null || recoveryParam === undefined) {
    throw new Error('签名失败：缺少 recovery 参数');
  }

  const v = (recoveryParam + 27).toString(16).padStart(2, '0');

  return `0x${r}${s}${v}`;
}

/**
 * 生成某个 DID 的私钥在微信本地存储中的键名。
 * 格式：didPrivateKey:<did>
```

```
 * @param {string} did - DID 字符串。
 * @returns {string}
 */
function getPrivateKeyStorageKey(did) {
  return `didPrivateKey:${did}`;
}

/**
 * 将指定 DID 的私钥保存到微信本地缓存。
 * 保存前会先对私钥格式做标准化处理。
 * @param {string} did - DID。
 * @param {string} privateKey - 私钥字符串。
 */
function saveDidPrivateKey(did, privateKey) {
  if (!did) {
    throw new Error('缺少DID，无法保存私钥');
  }

  if (!privateKey) {
    throw new Error('缺少私钥，无法保存');
  }

  wx.setStorageSync(getPrivateKeyStorageKey(did),
normalizePrivateKey(privateKey));
}

/**
 * 从微信本地缓存读取指定 DID 对应的私钥。
 * @param {string} did - DID。
 * @returns {string} 私钥字符串，不存在时返回空字符串。
 */
function getDidPrivateKey(did) {
  if (!did) return '';
  return wx.getStorageSync(getPrivateKeyStorageKey(did)) || '';
}

/**
 * 判断本地是否已经保存了某个 DID 的私钥。
 * @param {string} did - DID。
 * @returns {boolean}
 */
function hasDidPrivateKey(did) {
  return !!getDidPrivateKey(did);
}

/**
 * 从微信本地缓存中删除指定 DID 的私钥。
 * @param {string} did - DID。
 */
function removeDidPrivateKey(did) {
  if (!did) return;
  wx.removeStorageSync(getPrivateKeyStorageKey(did));
}

module.exports = {
  createDidKeyPair,
```

```
  getAddressFromPrivateKey,
  extractAddressFromDid,
  validatePrivateKeyForDid,
  signChallenge,
  saveDidPrivateKey,
  getDidPrivateKey,
  hasDidPrivateKey,
  removeDidPrivateKey
};
```

- **椭圆曲线 secp256k1**：和比特币、以太坊使用同一条曲线。私钥就是一个巨大的随机数，公钥由私钥通过曲线运算得出，但反过来几乎不可能，这就是安全基础。
- **以太坊地址**：由公钥做 keccak256 哈希后取最后 20 字节得到，通常用 EIP-55 校验和来包含大小写以降低输错概率。
- **personal_sign 签名**：为了安全签名，避免直接签名原始交易，以太坊引入了前缀规范 `\x19Ethereum Signed Message:\n<长度>`。这样签出来的数据可以明确区分于普通交易，后端用相关库恢复地址。
- **DID** `did:ethr`：是一种把以太坊地址包装成 DID 的方法，格式固定，方便对接 W3C 标准。

一、elliptic 库是做什么的?

elliptic 是一个纯 JavaScript 实现的椭圆曲线密码学库。
它不依赖 Node.js 的 `crypto` 模块，也不依赖浏览器插件，所以在微信小程序这种受限环境中也能正常运行。

最核心的能力只有两个：

1. **生成密钥对**（私钥 → 公钥）
2. **使用私钥对消息签名**（产生 r、s、v）

我们的代码只使用了椭圆曲线 `secp256k1`，这和比特币、以太坊是同一条曲线。

二、第一性原理：凭什么你可以证明“你是你”?

在数字世界，我们不能用指纹或人脸，只能用**数学秘密**。
这就需要一种“只有你知道，但别人可以验证”的东西。

非对称加密（也叫公钥密码学）恰好满足这一点：

- **私钥**：一个完全随机的巨大数字，只有你知道，打死也不能告诉别人。
- **公钥**：由私钥通过数学运算（椭圆曲线乘法）推导出来。这个过程**单向**：有私钥一定可以算出公钥，但从公钥反推私钥在计算上几乎不可能。
- **地址**：公钥再经过哈希运算，得到的一个短一点的代表身份的字符串（如以太坊地址）。

这三者的关系可以用一句话概括：

> **私钥生成公钥，公钥生成地址。**
> 你把地址公开说“这是我的 DID”，然后用私钥签名来证明你确实拥有这个地址背后的私钥，而不需要泄露私钥本身。

三、椭圆曲线签名到底在做什么?

想象你有一枚别人无法伪造的印章。
私钥就是印章本身，签名就是用印章盖的一个戳。
任何人都可以用你的公钥（很像印章的防伪特征）去验证这个戳是不是你的印章盖的，却没办法复制你的印章。

数学上就是三步：

1. 把要签名的消息（比如挑战字符串）通过哈希转换成一个数字 `h`。
2. 结合私钥 `d` 和一个随机数 `k`，用椭圆曲线运算算出两个值：**r** 和 **s**，以及一个恢复参数 **v**。
3. 把 `r + s + v` 拼在一起，就是你的签名。

验证时，对方拿到你的消息、签名、你的地址（或公钥），**不需要你的私钥**，就可以用椭圆曲线算法验证这个签名确实是由你的公钥对应的私钥产生的。

以太坊的 `personal_sign` 还在消息前面加了一个固定前缀 `\x19Ethereum Signed Message:\n<长度>`，让签出来的东西无法被用在其他场景（比如签交易），增加安全性。

四、完整流程拆解（对应代码）

我们以用户创建一个 DID，然后用它完成挑战-响应认证为例，把代码函数的执行顺序串起来。

1. 生成密钥对 `createDidKeyPair()`

```
createPrivateKey()  → 随机 32 字节私钥
getAddressFromPrivateKey()  → 由私钥推公钥 → 哈希 → 地址
拼接成 did:ethr:<地址>
```

具体代码步骤：

- `createPrivateKey()`
  用 `randomBytes(32)` 生成 32 个随机字节，拼成 64 位十六进制串。
  然后检查这个私钥数值是否在 `secp256k1` 的有效范围内（必须大于 0 小于曲线阶）。
  如果无效就再试，最多尝试 32 次。
- `getAddressFromPrivateKey(privateKey)`
  1. `getKeyPairFromPrivateKey` → 调用 elliptic 库，从私钥构造 keyPair 对象。
  2. `keyPair.getPublic(false, 'array')` → 获取未压缩公钥（65 字节：`04` + x 坐标 + y 坐标）。
  3. 去掉开头的 `04` 得到 64 字节，用 keccak256 哈希，取最后 20 字节作为地址。
  4. 通过 `toChecksumAddress` 变成 EIP-55 混合大小写地址（防输入错误）。

最终 `createDidKeyPair()` 返回 `{ privateKey, address, did }`，这就是用户的**完整数字身份**。

---

2. 存储私钥 `saveDidPrivateKey(did, privateKey)`

```
normalizePrivateKey → 统一成 0x + 64 位小写十六进制
wx.setStorageSync(`didPrivateKey:${did}`, 标准化私钥)
```

它只把私钥存在手机本地缓存里。
之后可以通过 `getDidPrivateKey(did)` 读取，`hasDidPrivateKey(did)` 判断是否存在，`removeDidPrivateKey(did)` 删除。

---

3. 挑战-响应签名 `signChallenge(privateKey, challenge)`

这是身份证明的核心：服务器发来一个随机挑战（比如 `"982156"`），你用私钥签名。

```
getEthereumMessageHash(challenge)   → 计算 personal_sign 格式的消息哈希
keyPair.sign(messageHashHex)        → elliptic 签名，得到 r, s, recoveryParam
拼接签名 = 0x + r(64位) + s(64位) + v(两位)
```

详细步骤：

1. `getEthereumMessageHash(challenge)`
   把挑战字符串转成 UTF-8 字节，并在前面拼接 `\x19Ethereum Signed Message:\n` 和消息字节长度，然后整个做 `keccak256` 哈希。
   这保证签名与以太坊生态兼容。
2. `keyPair.sign(messageHashHex, { canonical: true })`
   使用 secp256k1 椭圆曲线对哈希签名。`canonical: true` 是为了防止签名可锻性（一个消息可以有多个合法签名，选一个规范形式）。
3. **提取 r, s, recoveryParam**
   - `r` 和 `s` 是椭圆曲线签名中的两个大整数，各 32 字节，补零到 64 位十六进制。
   - `recoveryParam` 是一个 0 或 1 的数，之后 `v = recoveryParam + 27`。`v` 用于从签名和消息中恢复出签名者的公钥（或地址），这是以太坊特有的。

最终返回的签名是一个 130 个字符的十六进制字符串（`0x` + 64 位 r + 64 位 s + 2 位 v）。
把这个签名发给服务器，服务器调用类似 `eth_account.recover_message` 就能得到签名者的以太坊地址。
如果这个地址与用户 DID 中的地址一致，就认证通过。

五、为什么要这样设计?

目标	实现方式
不依赖网络也能创建身份	完全本地生成私钥，随机数用设备安全源或 Math.random 兜底
身份与区块链挂钩	`did:ethr` 标识符直接对应以太坊地址，链上可查
证明身份而不泄露私钥	签名使用椭圆曲线，服务器只能拿到签名和地址，无法反推私钥
兼容以太坊生态	完全遵循 `personal_sign` 规范，签名可以被标准工具验证

目标	实现方式
轻量化、适应小程序	用 `elliptic` 而不是 `ethers`，避开 Buffer 等依赖，体积小

六、一张图总结整个加密流程

